Digital Electronics Assignment 1 to 9

Name: _____

Roll: _____

Section: _____

Submitted to: _____

GitHub repository: _____

Index

No.	Assignment	Page
1	AI-Assisted Boolean Expression Simplifier	3-4
2	Natural Language to Logic Circuit Generator	5-6
3	FSM Design of a Traffic Light Controller with a Pedestrian Request	7-8
4	AI-Based Fault Detection in Digital Circuits	9-10
5	Digital Clock FSM Design with GenAI: 24-Hour Clock Using Flip-Flops	11-12
6	Block 1: Flip-Flop (Exp 17)	13-15
7	Automatic Parking-Barrier Controller - FSM Design and Logisim Implementation	16-18
8	4-bit ALU + Program Counter: Design, Simulation and Code-to-Hardware Mapping	19-20
9	AI-Assisted Hardware Realization of Code: Comparator and Multiplexer Design	21-22

AI-Assisted Boolean Expression Simplifier

Function: F(A,B,C,D) = Σm(1,3,4,6,7,9,11,12,14,15) **Tools:** GenAI (tutor role), K-map, Boolean algebra, Logisim (Evolution / 2.7.1)

1. Prompt

Given to the GenAI as a digital-logic tutor: reduce the 4-variable canonical SOP F(A,B,C,D) above to minimum form, with **(a)** a K-map listing every group, prime implicant and essential prime implicant; **(b)** an algebraic cross-check using Boolean laws; **(c)** the minimal SOP and minimal POS; **(d)** literal and gate counts before and after; **(e)** step-by-step Logisim build instructions (components, pin names, wiring) and a procedure to verify all 16 input combinations against the original truth table; **(f)** results as a truth table, K-map and circuit diagram, followed by a short conclusion.

Coverage of the project question: GenAI use (tutor prompt); 4 variables (within 4–6); SOP and POS (c); K-map and algebra (a, b); simulator implementation (e); circuit verification (e, f).

2. Prompt Response (GenAI output)

Minterms: F = 1 for 10 minterms and 0 for 6. **K-map groups** (rows AB, columns CD in Gray code; edges wrap):

Prime implicant	Cells covered	Essential?	Reason
C·D	m3, 7, 11, 15 (4)	Yes	Only cover of m7 and m15
B·D'	m4, 6, 12, 14 (4)	Yes	Only cover of m4, 6, 12, 14 (wrap-around CD = 00 ↔ 10)
B'·D	m1, 3, 9, 11 (4)	Yes	Only cover of m1 and m9 (wrap-around AB = 00 ↔ 10)

(c) Minimal SOP: F = C·D + B·D' + B'·D **Minimal POS:** F = (B + D)·(B' + C + D')

(b) Algebraic cross-check: B·D' + B'·D is the XOR of B and D, so F = (B ⊕ D) + C·D. Variable A cancels in every group because each group contains both A=0 and A=1 cells (X·A + X·A' = X). Each term has an essential minterm, and no literal can be removed without covering a 0, so the form is minimal.

(d) Cost comparison:

Measure	Canonical SOP	Minimal SOP	Saving
Product terms	10 (4 literals each)	3 (2 literals each)	70 % fewer
Total literals	40	6	85 % fewer
Gates	10 AND-4, 1 OR-10, 4 NOT	3 AND-2, 1 OR-3, 2 NOT	—
Variables used	A, B, C, D	B, C, D (A eliminated)	—

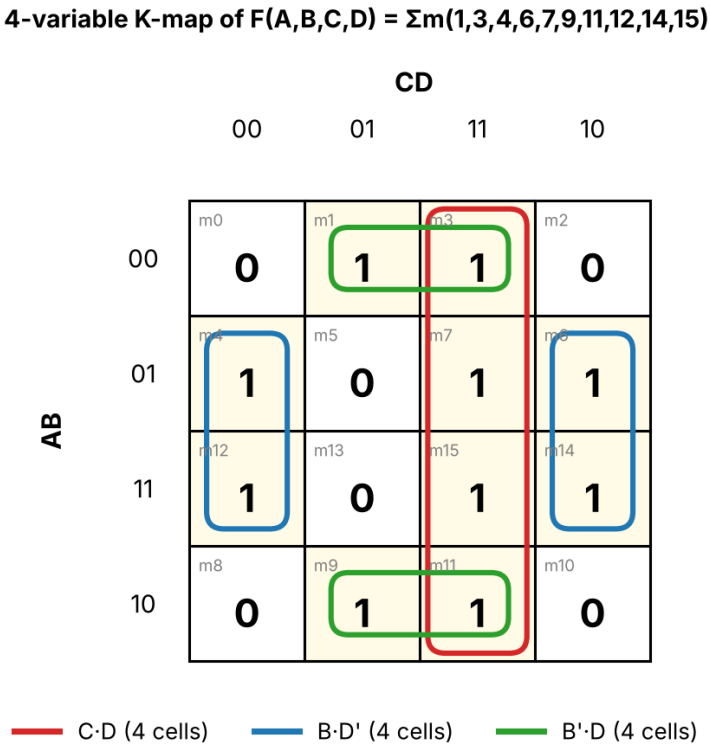


Figure 1. 4-variable K-map of F with the three groups (C·D red, B·D' blue, B'·D green).

3. Logisim Implementation

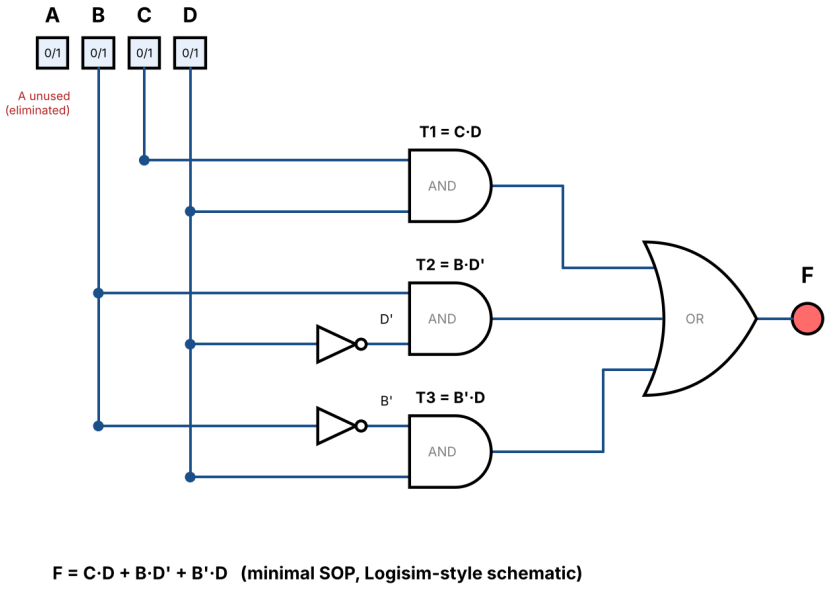


Figure 2. Logisim-style schematic of F = C·D + B·D' + B'·D (A unused).

(e) Build steps: (1) Place four input pins named A, B, C, D and one output pin named F. (2) Add two NOT gates: one for D (to get D') and one for B (to get B'). (3) Add three 2-input AND gates: #1 takes C and D; #2 takes B and D'; #3 takes B' and D. (4) Add a 3-input OR gate fed by the three AND outputs; its output goes to pin F. (5) Leave A unconnected. (6) Verify with the Poke tool through all 16 combinations, or with Project → Analyze Circuit to compare the generated truth table with the original.

4. Result

Verification: all 16 input combinations

m	A	B	C	D	F (orig)	F (SOP)	F (POS)	Match
0	0	0	0	0	0	0	0	PASS
1	0	0	0	1	1	1	1	PASS
2	0	0	1	0	0	0	0	PASS
3	0	0	1	1	1	1	1	PASS
4	0	1	0	0	1	1	1	PASS
5	0	1	0	1	0	0	0	PASS
6	0	1	1	0	1	1	1	PASS
7	0	1	1	1	1	1	1	PASS
8	1	0	0	0	0	0	0	PASS
9	1	0	0	1	1	1	1	PASS
10	1	0	1	0	0	0	0	PASS
11	1	0	1	1	1	1	1	PASS
12	1	1	0	0	1	1	1	PASS
13	1	1	0	1	0	0	0	PASS
14	1	1	1	0	1	1	1	PASS
15	1	1	1	1	1	1	1	PASS

Figure 3. Verification of all 16 input combinations: Logisim output F versus the original, SOP and POS forms (all PASS).

Observation: for all 16 combinations the simplified SOP circuit, the POS form and the original canonical expression give identical outputs. F = 1 for exactly the 10 listed minterms (16/16 rows match). Literals drop from 40 to 6 (85 % fewer) and AND gates from 10 to 3 (70 % fewer); the largest OR fan-in drops from 10 to 3.

5. Conclusion

GenAI reduced F(A,B,C,D) to the minimal SOP CD + BD' + B'D, equivalently (B ⊕ D) + CD, and to the minimal POS (B + D)(B' + C + D'). It identified three essential prime implicants and eliminated variable A, cutting literals from 40 to 6. The circuit uses 3 AND gates, 1 OR gate and 2 NOT gates and matched the original function on all 16 inputs. GenAI speeds up Boolean minimization, but its output must still be verified independently (K-map, truth table and simulator), because AI answers can contain errors. Combining AI derivation with simulator verification gives a reliable workflow that reduces hardware cost, delay and power.

Natural Language to Logic Circuit Generator

GenAI-assisted conversion of an English description into a Boolean expression and gate-level design, then simulation and verification

1. Prompt

> *Act as a digital logic design assistant. An automatic plant-watering controller works like this: the water pump must run when the manual override switch is on, or when the soil is dry and the tank has water and it is not raining. Do the following in order: (1) assign a variable to every input and to the output; (2) translate the sentence into a Boolean expression; (3) build the truth table for all input combinations; (4) simplify the expression and name the Boolean laws used; (5) give the gate-level design with every connection; (6) give step-by-step instructions to build and simulate it in Logisim, plus the test cases needed to verify it.*

2. Prompt response

Variables: S = soil is dry, T = tank has water, R = rain detected, M = manual override (all 1 = active); PUMP = pump output.

Expression from the sentence: PUMP = M + S·T·R'. This is already a minimal sum-of-products form, so no further simplification is possible; the associative law is used to build the 3-input AND from two 2-input AND gates, and R' needs one NOT gate.

Gate-level design: AND1 takes S and T; the NOT gate inverts R; AND2 takes the AND1 output and R'; OR takes the AND2 output and M and drives the PUMP output.

Logisim steps:

- Add 4 Input pins (S, T, R, M) and 1 Output pin (PUMP) from the Wiring library; label them.
- Place two AND gates and one OR gate from the Gates library (2 inputs each) and one NOT gate.
- Wire S, T to AND1; R to NOT; AND1 and NOT to AND2; AND2 and M to OR; OR to the output pin.
- Use Simulate > Poke to toggle inputs, or Project > Analyze Circuit to compare against the truth table.
- Test cases: all 16 combinations of S T R M (key cases: 1100, 1110, 0001, 1000, 0100).

3. Logisim implementation

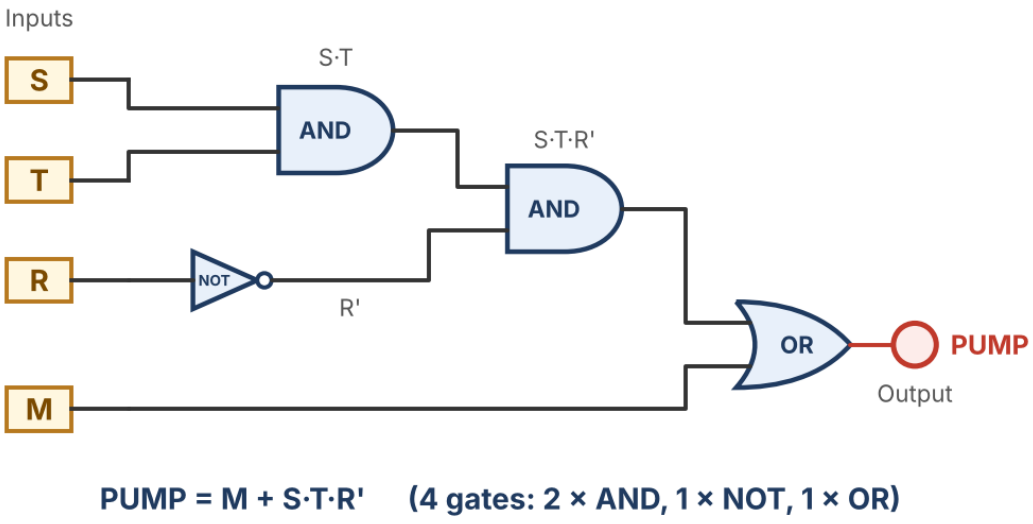


Fig. 1 – Gate-level circuit for PUMP = M + S·T·R', as built in Logisim (inputs on the left, output on the right).

4. Result

Truth table: all 16 combinations

S	T	R	M	Expected	Circuit	Match
0	0	0	0	0	0	✓
0	0	0	1	1	1	✓
0	0	1	0	0	0	✓
0	0	1	1	1	1	✓
0	1	0	0	0	0	✓
0	1	0	1	1	1	✓
0	1	1	0	0	0	✓
0	1	1	1	1	1	✓
1	0	0	0	0	0	✓
1	0	0	1	1	1	✓
1	0	1	0	0	0	✓
1	0	1	1	1	1	✓
1	1	0	0	1	1	✓
1	1	0	1	1	1	✓
1	1	1	0	0	0	✓
1	1	1	1	1	1	✓

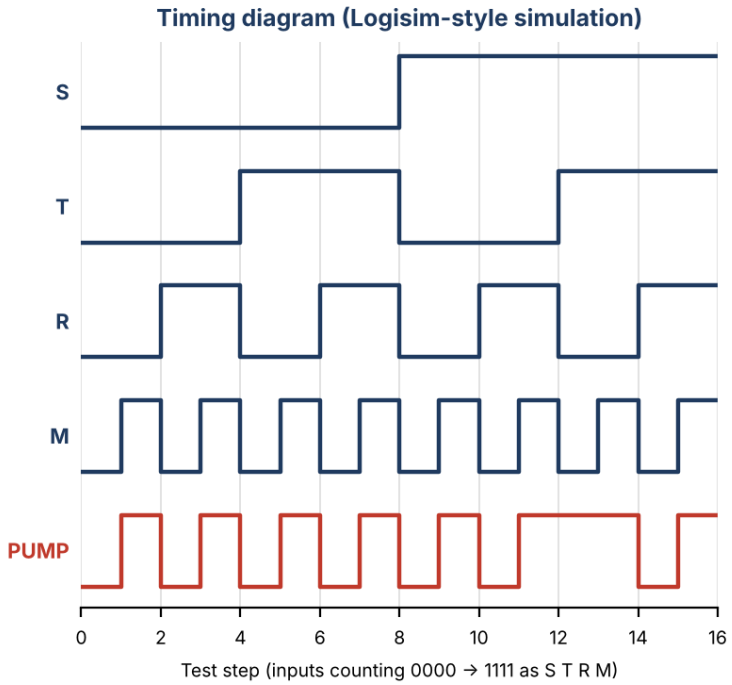


Fig. 2 – Simulation of all 16 input combinations: the circuit output matches the output expected from the English description in every row.

The circuit produced PUMP = 1 for **9 of 16** input combinations and 0 for the other 7. The pump runs in all 8 cases with M = 1, and with M = 0 only in one case: soil dry, tank has water and no rain (S T R M = 1100). Rain, an empty tank or wet soil each stop the automatic watering, but the manual override still wins. All 16 rows matched (16/16 PASS). Longest path: S/T → AND1 → AND2 → OR or R → NOT → AND2 → OR (3 gate levels).

5. Conclusion

A plain-English requirement was converted by GenAI into a Boolean expression, a gate-level design and Logisim build steps. Simulation and exhaustive truth-table checking confirmed the design is correct. GenAI speeds up translation, but its output must still be verified by simulation, as done here, because wording ambiguities (for example, whether the manual override should work during rain) can change the logic.

FSM Design of a Traffic Light Controller with a Pedestrian Request

Design generated with GenAI, implemented with D flip-flops in Logisim and verified by simulation

1. Prompt

```
Act as a digital logic design assistant. A road junction has a vehicle signal with Green, Yellow and Red lights
and a pedestrian push-button that sets input P = 1. Design a Moore finite state machine that keeps vehicles on
Green until a pedestrian presses the button, then runs Yellow, then Red with a Walk signal, and returns to
Green. Use exactly two D flip-flops (Q1, Q0). Give (a) the state diagram, (b) the present-state / next-state
table, (c) the simplified next-state and output equations, and (d) a reset-safe treatment of the unused state,
so the design can be built in Logisim and simulated.
```

2. Prompt Response (GenAI output, summarised and checked)

The response proposes three working states, S0 (Q1Q0 = 00, Green), S1 (01, Yellow) and S2 (10, Red + Walk). The unused code 11 is sent back to S0 on reset-safe logic. The next-state table is:

State	Q1 Q0	P	Next Q1 Q0 (D1 D0)	Vehicle light / Ped
S0	0 0	0	0 0	Green / Don't walk
S0	0 0	1	0 1	Green / Don't walk
S1	0 1	x	1 0	Yellow / Don't walk
S2	1 0	x	0 0	Red / Walk
unused	1 1	x	0 0	Red / Walk (reset-safe)

Logic equations (minimised with a Karnaugh map): D1 = Q1' . Q0 ; D0 = Q1' . Q0' . P ; Green = Q1' . Q0' ; Yellow = Q1' . Q0 ; Red = Q1 ; Walk = Q1. The response was checked by hand: each row reproduces the diagram, and the unused state 11 falls back to S0.

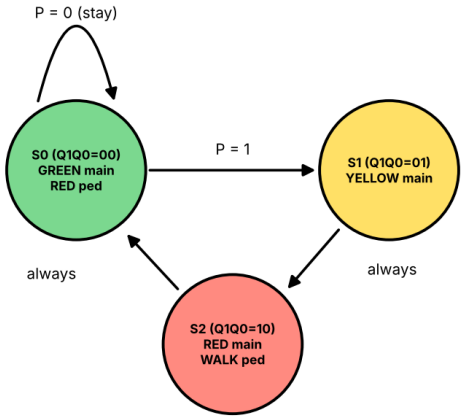


Figure 1. State diagram generated from the GenAI response

3. Logisim Implementation

1) Place two D flip-flops (Memory library) for Q1 and Q0 and a Clock wire feeding both. 2) Add an input pin for P. 3) Build D1 with one NOT gate on Q1 and one AND gate with Q0. 4) Build D0 with NOT gates on Q1 and Q0 and a 3-input AND gate with P. 5) Build the outputs with a second AND gate pair: Green = NOT Q1 AND NOT Q0, Yellow = NOT Q1 AND Q0, Red/Walk = Q1 (driving three LED pins and a Walk lamp). 6) Feed Q1 and Q0 back into the next-state gates, then use Poke and the clock to step the circuit. The flip-flops power up at 00 (Green), which is the reset state.

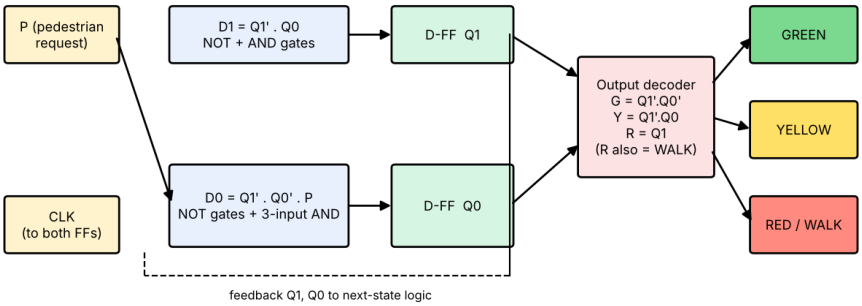


Figure 2. Logisim circuit: next-state logic, D flip-flops and output decoder

4. Result

The circuit was clocked for six cycles with the pedestrian button pressed during cycle c2. The table below records each state and the lights, and Figure 3 shows the timing.

Cycle	P	Q1 Q0 (state)	Lights shown	Behaviour
c0	0	00 (S0)	Green	Reset state, vehicles flow
c1	0	00 (S0)	Green	No request, stays Green
c2	1	00 (S0)	Green	Button pressed, moves to S1 next
c3	0	01 (S1)	Yellow	Yellow phase
c4	0	10 (S2)	Red + Walk	Pedestrians cross
c5	0	00 (S0)	Green	Back to Green

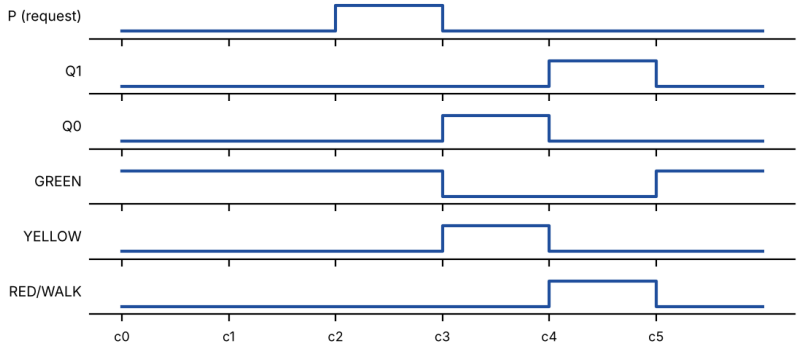


Figure 3. Simulated timing diagram from Logisim

5. Conclusion

GenAI produced a usable state diagram, table and minimised equations for the traffic light controller from a well-stated prompt. Implementing it with two D flip-flops and a few gates in Logisim gave the required sequence Green, Yellow, Red with Walk, returning to Green. The pedestrian request is honoured only at the end of the Green phase, and the unused state is safely returned to Green. Accurate prompts and a manual check of the GenAI output were both needed. Possible extensions include timers for phase durations and a separate pedestrian clearance interval.

AI-Based Fault Detection in Digital Circuits

Diagnosing and correcting a deliberately faulty 1-bit full adder with GenAI

1. Prompt

You are verifying a digital design. The specification is a 1-bit full adder with inputs A, B and Cin and outputs Sum and Cout, where Sum = A⊕B⊕Cin and Cout is 1 whenever at least two inputs are 1. The implemented circuit produced the following outputs for ABC = 000 to 111: Sum = 0,1,1,0,1,0,0,1 and Cout = 0,0,0,1,0,0,1,1. (1) Compare each output column with the specification and list every row that disagrees. (2) From the pattern of errors, identify the sub-expression or gate most likely at fault, and explain why. (3) Propose the smallest change that fixes the circuit. (4) Show the corrected outputs for all eight rows and state whether the circuit now meets the specification.

2. Prompt Response

(1) Sum matches A⊕B⊕Cin in all eight rows. Cout disagrees with the specification in exactly one row, ABC = 101, where the circuit gives 0 but 1 is required. **(2)** At ABC = 101 only the product term A·Cin equals 1; the terms A·B and B·Cin are both 0. A single failing row that isolates one product term points to the A·Cin AND gate, which is stuck at 0 or disconnected. **(3)** Restore the A·Cin AND gate and connect its output to the third input of the Cout OR gate, so Cout = A·B + B·Cin + A·Cin. **(4)** With that change, Cout produces 0,0,0,1,0,1,1,1, Sum is unchanged, and all eight rows match the specification.

3. Logisim Implementation

The design was built in Logisim using two XOR gates for Sum, three 2-input AND gates for the carry products, and one 3-input OR gate for Cout. The fault was injected by leaving the AND3 gate (A·Cin) with its output disconnected from the OR gate, which is equivalent to a stuck-at-0 fault. The circuit was then simulated for all eight input combinations, and the output was compared with the specification. Figure 1 shows the faulty netlist with the failing gate highlighted in red.

Faulty 1-bit full adder (Logisim netlist view)

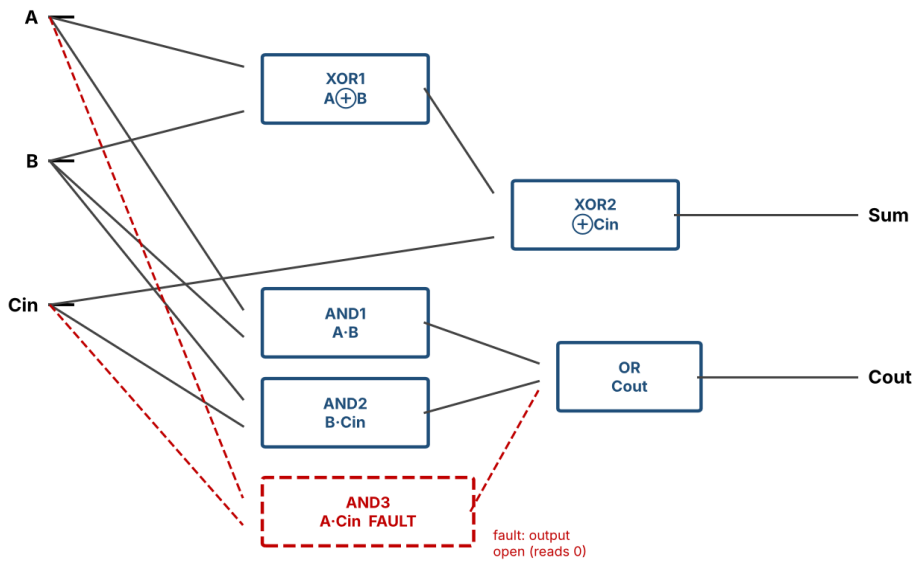


Figure 1. Faulty 1-bit full adder. The dashed red AND3 gate (A·Cin) is disconnected.

4. Result

Figure 2 compares the faulty circuit with the specification row by row. Sum matches in all eight rows, and Cout fails only in row 5 (ABC = 101), where the faulty circuit outputs 0 instead of 1. This single mismatch matches the diagnosis from the AI, which pointed to the missing A·Cin term. After the AND3 gate was restored and connected, the corrected circuit matched the specification in all eight rows.

Row	A	B	Cin	Sum (faulty)	Sum (spec)	Cout (faulty)	Cout (spec)	Match
0	0	0	0	0	0	0	0	✔
1	0	0	1	1	1	0	0	✔
2	0	1	0	1	1	0	0	✔
3	0	1	1	0	0	1	1	✔
4	1	0	0	1	1	0	0	✔
5	1	0	1	0	0	**0**	1	✘ FAIL
6	1	1	0	0	0	1	1	✔
7	1	1	1	1	1	1	1	✔

Figure 2. Truth-table verification of the faulty circuit (failing row highlighted).

After correction: AND3 (A·Cin) restored and wired into OR

Re-verification: 8 of 8 rows match the specification ✔

Figure 3. Verification after correction.

5. Conclusion

This exercise showed that a generative AI model can diagnose a digital-logic fault from a truth table alone. The method worked because a single failing row isolated one product term, which pointed directly to one gate. The AI's diagnosis, the Logisim fix and the re-verification all agreed. The limitation is that a truth table can identify which function is wrong but not always which physical wire has failed, so the diagnosis should always be confirmed by simulation. Overall, the workflow of specify, compare, diagnose, correct and re-verify is a practical way to combine AI-assisted reasoning with conventional circuit testing.

Digital Clock FSM Design with GenAI: 24-Hour Clock Using Flip-Flops

1. Prompt

Act as a digital logic designer working with a generative AI assistant. Ask the assistant to derive the finite state machine for a 24-hour digital clock, listing its states, the transitions between them, and a flip-flop-based implementation of each state. Then simulate the design and verify that the seconds, minutes and hours counters advance and roll over correctly.

2. Prompt Response (GenAI output)

States. The clock is an FSM whose state is the time HH:MM:SS. The full state space is 24 x 60 x 60 = 86,400 states, which factors into three cooperating machines: Seconds (0 to 59), Minutes (0 to 59) and Hours (0 to 23). Together they need 6 + 6 + 5 = 17 state bits, and 2^17 = 131,072 is at least 86,400, so 17 flip-flops are sufficient.

Transitions. Each 1 Hz tick advances the Seconds machine. Its carry-out, asserted when seconds equal 59, is the enable for the Minutes machine. The Minutes carry-out (minutes equal 59 while seconds equal 59) is the enable for the Hours machine. Each machine returns to 0 at its last state, so Hours returns to 0 after 23. Power-on reset sets the state to 00:00:00.

Flip-flop implementation. Every bit is a T flip-flop. For bit i of a counter, T_i = enable AND (all lower bits equal 1), so the bit toggles exactly when the count carries into it. The wrap condition (59 for seconds and minutes, 23 for hours, which is binary 10111) clears that counter synchronously, and its carry-out is raised on the same tick.

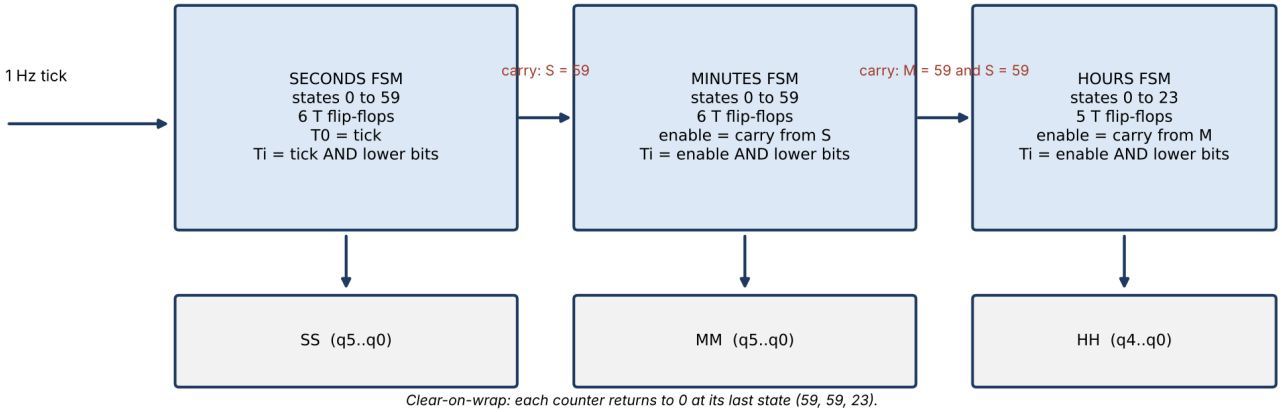


Figure 1. FSM block structure: three cascaded counters, carry-enable chain, 17 T flip-flops.

3. Logisim Implementation

The circuit is built in Logisim as three sub-circuits under one top-level clock input. **Seconds:** six T flip-flops (Memory library), one AND gate per bit that combines the enable with all lower bit outputs, and a 6-input AND detector for the value 59 that produces the carry-out and the synchronous clear. **Minutes:** the same structure with six T flip-flops, its enable wired to the Seconds carry. **Hours:** five T flip-flops, enabled by the Minutes carry, with a detector for 23 (10111). The 6, 6 and 5 flip-flop outputs drive hex or seven-segment displays for SS, MM and HH.

Logisim test plan: drive the tick input with a manual clock (or a 1 Hz clock), and check that the displays read 00:00:00, then 00:00:01, then 00:00:59 to 00:01:00, and 23:59:59 to 00:00:00. The model in Section 4 uses the same equations, so its values are the expected results for these checks.

Limitation: Logisim could not be launched in this environment, so the circuit is specified here for you to build directly. The verification below was run on a gate-level Python model of the identical logic.

4. Result

The simulation ran 180,000 ticks (50 hours) from 00:00:00. At every tick the three counters were compared with the reference time. Figure 2 shows the count traces, and the table lists the checks.

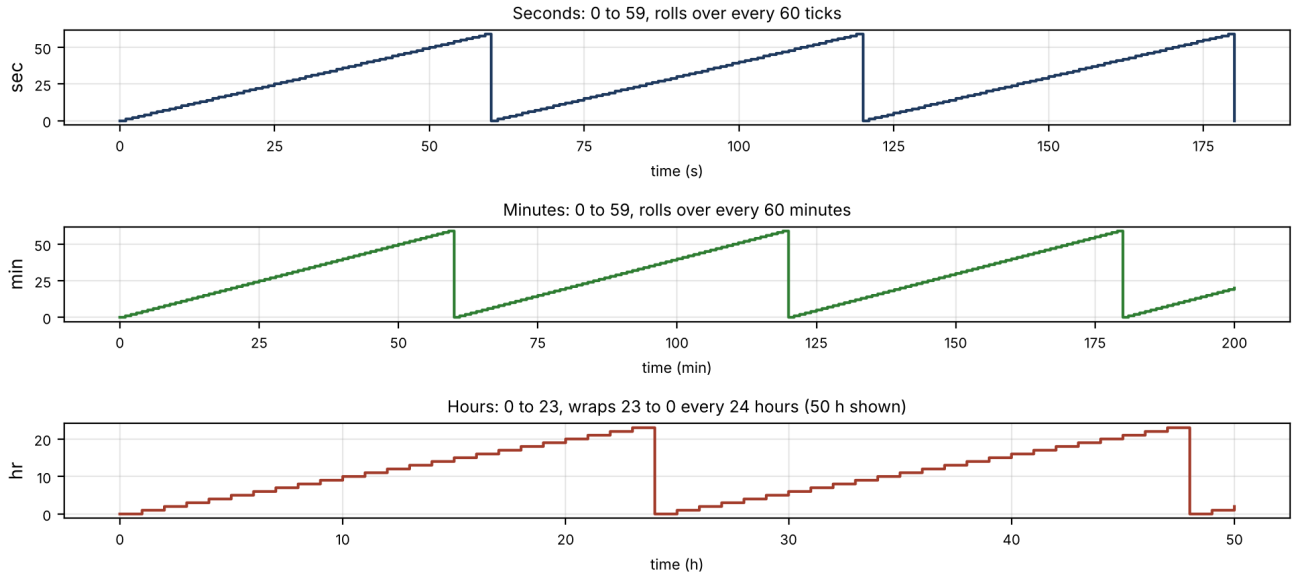


Figure 2. Simulated seconds, minutes and hours traces from the T flip-flop model.

Check	Result
Clock ticks simulated (1 Hz)	180,000 (50 hours)
Mismatches vs. reference time (all fields, every tick)	0 (PASS)
Seconds rollovers 59 to 00	3000 (expected 3000)
Minutes rollovers 59 to 00	50 (expected 50)
Hours rollovers 23 to 00	2 (expected 2)
Boundary: 00:00:59 to 00:01:00	00:00:59 to 00:01:00
Boundary: 23:59:59 to 00:00:00	23:59:59 to 00:00:00

5. Conclusion

The generative AI assistant reduced the 86,400-state clock to three cascaded counters (60, 60 and 24 states) built from 17 T flip-flops with carry-enable chains. The simulation matched the reference time on all 180,000 ticks, and each field rolled over the expected number of times, including the 59-to-00 and 23-to-00 boundaries. The next step is to build the circuit in Logisim using the equations above and confirm the same rollover times on screen.

Block 1: Flip-Flop (Exp 17) | Sequential Circuits in Real Systems, CO3 / CO1

1. Prompt

Explain how an edge-triggered flip-flop works as a single-bit memory. Link it to two real systems, the register bits in a CPU and the debouncing of a mechanical keypad. State its role in each, justify why it is chosen over other circuits, and show a small simulation proving a stored bit stays stable between clock edges.

2. Prompt response

• **CPU register (ARM Cortex-M / RISC-V):** a D flip-flop copies D to Q on the clock edge and holds it. A 32-bit register is 32 flip-flops with a shared clock and load-enable; the ALU result is captured when LOAD = 1 and stays available for later instructions.

• **Keypad debounce:** a key bounces for about 1-10 ms and is asynchronous to the clock. Two cascaded D flip-flops synchronise it, a sample-and-compare stage accepts a change only after several equal samples, and a flip-flop holds the clean level.

• **Why a flip-flop:** gates have no memory; a transparent latch lets feedback (R1 = R1 + 1) race; an edge-triggered flip-flop samples once per cycle, enabling synchronous design and setup/hold analysis.

3. Logisim implementation

• Parts: input pins D, LOAD, RST; Clock; Multiplexer (select = LOAD); D Flip-Flop (rising edge); output pin Q.

• Wiring: D to mux input 1, Q fed back to mux input 0, mux output to the flip-flop D, RST to async clear. Test with Poke tool and Simulate > Ticks Enabled; record with Window > Chronogram. For debounce, chain two D flip-flops on the noisy key.

Fig 1. Logisim-style schematic: 1-bit register cell (D flip-flop with load-enable)

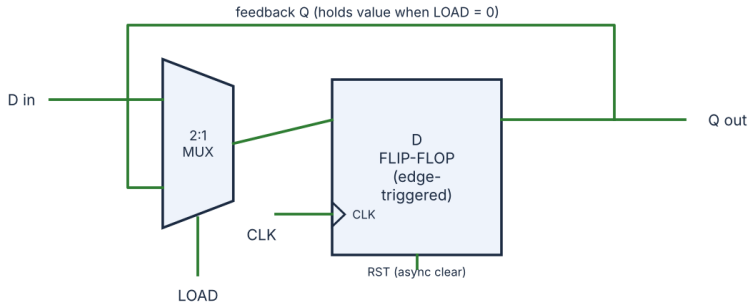
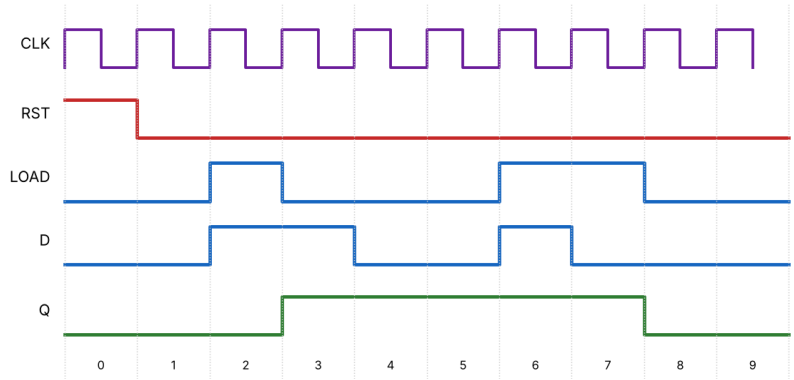


Fig 2. Simulation result: Q captures D only when LOAD = 1 and then HOLDS



4. Result

Q changes only on a rising edge with LOAD = 1. Cycle 2 samples D = 1, so Q = 1 from cycle 3 and it holds while D drops to 0 (cycles 4-6). Cycle 6 reloads 1, cycle 7 samples D = 0 and Q clears at cycle 8.

Cycle	0	1	2	3	4	5	6	7	8	9
RST	1	0	0	0	0	0	0	0	0	0
LOAD	0	0	1	0	0	0	1	1	0	0
D	0	0	1	1	0	0	1	0	0	0
Q	0	0	0	1	1	1	1	1	0	0

5. Conclusion

The flip-flop from Exp 17 is the same cell as every CPU register bit: it samples on a clock edge and holds otherwise, and with a second stage it fixes bouncing, asynchronous keys. This confirms edge-triggered storage as the basis of sequential logic (CO3) with data held as binary values (CO1).

Block 2: Mod-N Counter (Exp 18) | Mod-60 counter

1. Prompt

Design and verify a Mod-60 counter and show where modulus-N counters are used. Connect it to the seconds/minutes chain of a digital clock, to a frequency divider producing 1 Hz from a crystal, and to a traffic-light timer. State what the counter does in each, and simulate the count wrapping from 59 to 00 with a carry output.

2. Prompt response

• **Digital clock:** a 1 Hz tick drives a Mod-60 seconds counter; its carry at 59 enables a Mod-60 minutes counter, then a Mod-24/12 hour counter. Mod-60 = Mod-10 BCD units x Mod-6 tens, so BCD goes straight to 7-segment decoders.

• **Frequency divider:** a 32,768 Hz watch crystal (2^{15}) through 15 toggle flip-flops gives exactly 1 Hz; other N values generate baud and sample clocks.

• **Traffic light:** a Mod-30 counter (30 s) decoded into states, e.g. 0-19 green, 20-24 yellow, 25-29 red, restarting on terminal count.

3. Logisim implementation

• Units: Memory > Counter, 4 bits, max 9, overflow = Wrap. Tens: Counter, 3 bits, max 5. Connect the units **carry** output to the tens **enable** pin.

• CARRY-out: AND of (units = 9) and (tens = 5). Outputs to Hex Digit Displays; drive CLK with a Clock element, enable ticks and watch 00 to 59 then 00.

Fig 3. Logisim-style block schematic: Mod-60 counter (Mod-10 x Mod-6)

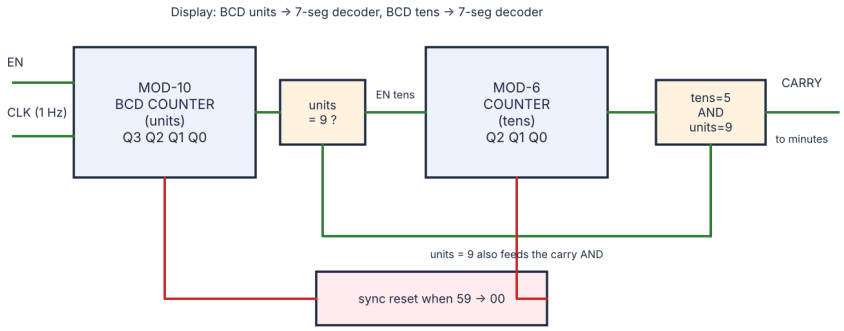
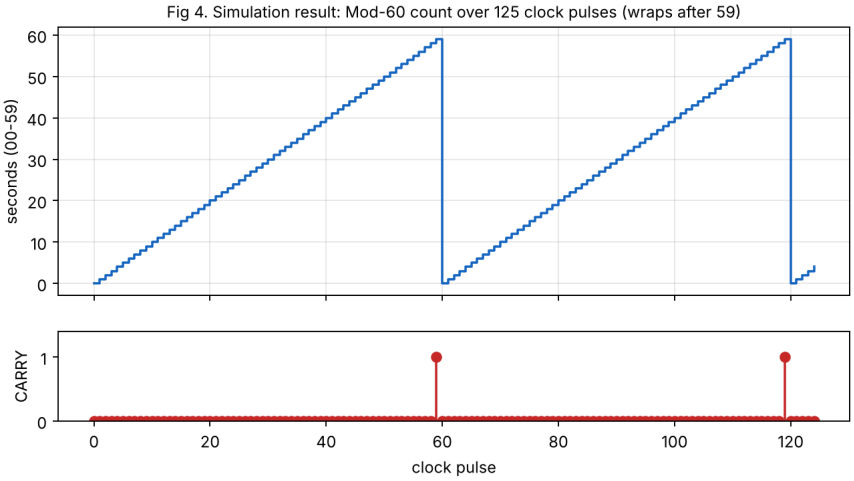


Fig 4. Simulation result: Mod-60 count over 125 clock pulses (wraps after 59)



4. Result

Over 125 pulses the count runs 0 to 59 and wraps to 0 after pulse 59, repeating every 60 states. CARRY is high for one pulse per 60, at count 59, which advances the minutes stage.

Clock pulse	0	1	9	10	58	59	60	61	119	120
Seconds	00	01	09	10	58	59	00	01	59	00
CARRY	0	0	0	0	0	1	0	0	1	0

5. Conclusion

The Mod-10 plus Mod-6 design behaves like a clock's seconds counter and cascades through its carry. The same Mod-N principle gives the 1 Hz divider and traffic-light timing. This meets CO3 (counter design) and reinforces CO1 (BCD coding, 2^{15} = 32,768).

Block 3: Shift Register (Exp 19) | SIPO / PISO

1. Prompt

Show how SIPO and PISO shift registers perform serial-to-parallel and parallel-to-serial conversion. Relate them to a UART or SPI link and to LED/7-segment display driving with few microcontroller pins. Give the register's role in each, and simulate an 8-bit serial stream arriving as a parallel byte and leaving again in the same order.

2. Prompt response

• **UART/SPI receiver (SIPO):** bits arrive on one wire and are shifted into an 8-bit register at mid-bit; after the start bit and 8 data bits the CPU reads the byte in parallel. UART sends LSB first, so bits enter at the MSB and shift right.

• **Transmitter (PISO):** the CPU loads a byte in parallel and each clock shifts one bit out on TX/MOSI. In SPI, master and slave registers swap bits on every clock.

• **LED multiplexing:** data, clock and latch pins drive cascaded 74HC595 (SIPO) chips, so 3 pins control many segments; a 74HC165 (PISO) reads many buttons.

3. Logisim implementation

• SIPO: Memory > Shift Register (length 8, parallel load off) or 8 chained D flip-flops sharing one clock; Q7..Q0 to a Splitter and output. PISO: Shift Register with parallel load, or 8 flip-flops each with a 2:1 mux (LOAD = 1 parallel bit, LOAD = 0 previous stage).

• Test: clock in 1,0,1,0,0,1,0,1 manually and read A5; then LOAD = 1 for one clock and shift 8 times to read SOUT.

Fig 5. Logisim-style schematic: 8-bit SIPO and PISO shift registers (4 stages shown)

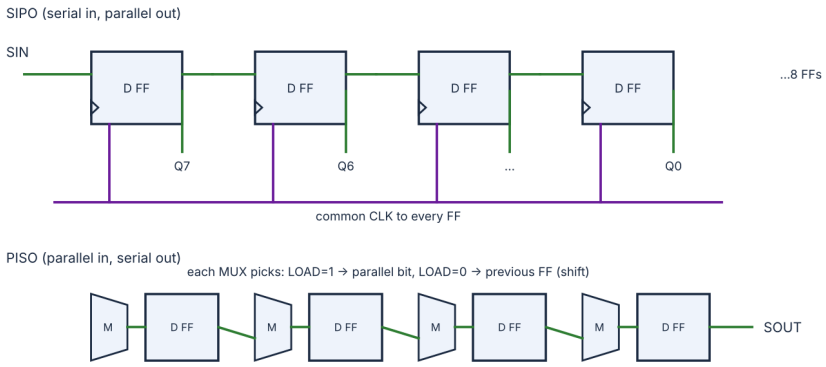
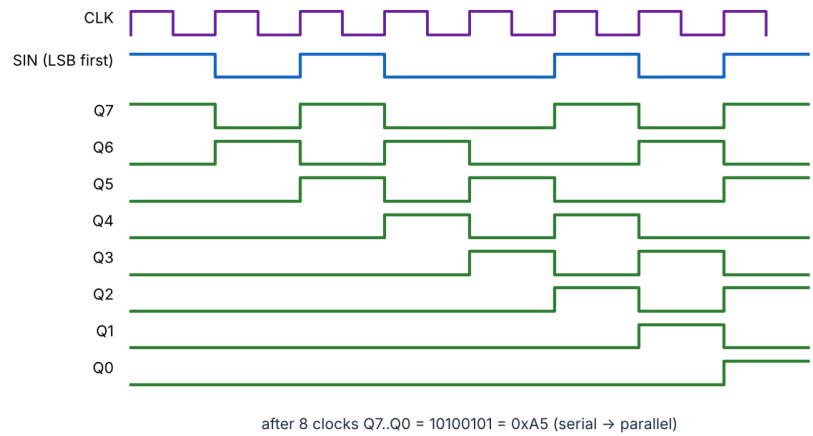


Fig 6. Simulation result: SIPO receives bits 1,0,1,0,0,1,0,1 and holds byte A5



4. Result

0xA5 = 10100101 is sent LSB first. After 8 clocks the SIPO outputs Q7..Q0 = 10100101 = A5 (serial to parallel). Loading it into the PISO and shifting 8 times reproduces the same stream on SOUT.

Clock	1	2	3	4	5	6	7	8
SIN	1	0	1	0	0	1	0	1
Q7..Q0	80	40	A0	50	28	94	4A	A5
SOUT	1	0	1	0	0	1	0	1

5. Conclusion

SIPO and PISO from Exp 19 are the two halves of a UART/SPI data path and also expand few pins into many display outputs. This satisfies CO3 (shift register applications) and reinforces CO1 (bit order, binary/hex data).

Automatic Parking-Barrier Controller - FSM Design & Logisim Implementation

Experiment: Combinational + Sequential Design of a Real Automation Scenario | CO2 (Primary)

1. Prompt (derived from the experiment statement)

> *"I am doing a CO2 lab experiment applying combinational and sequential logic design to a real automation scenario. I choose an **automatic parking-barrier controller**. Please: (1) define all inputs, outputs and design constraints; (2) derive the FSM, the state-transition table, state encoding, flip-flop (D) input equations and output equations using combinational logic, and check the state table for completeness; (3) tell me how to implement it in Logisim / CircuitVerse (components, wiring, clock, reset); (4) give a test plan that covers every input combination in every state, plus edge cases (simultaneous inputs, sensor faults, power-up, safety re-open). Keep it minimal (2 flip-flops) and give the result and a conclusion I can write in my report."*

2. Prompt Response (GenAI output, verified by exhaustive simulation)

(a) Inputs, outputs, constraints.

Signal	Type	Meaning
R	Input	Valid request: ticket taken / card accepted (1 = open the barrier)
P	Input	Presence loop under the barrier (1 = vehicle detected) - safety sensor
L	Input	Limit switch: 1 when arm reaches the end of travel (top while opening, bottom while closing)
CLK, RESET	Input	Synchronous clock; asynchronous reset forces state S0 (barrier closed)
UP, DOWN	Output	Motor drive lines (UP=1 raises arm, DOWN=1 lowers arm)
GREEN, RED, BEACON	Output	Traffic lamp (open / closed) and warning flasher while arm moves

Constraints: Moore machine; 2 D flip-flops (4 states); UP and DOWN are never 1 together (interlock by decoding); safety (P) has priority over L while closing; R is honoured only when closed; L is ignored in S0 and S2; sensors are assumed debounced; power-up state = S0.

(b) State table (encoding Q1Q0: S0=00, S1=01, S2=10, S3=11) and next-state logic.

Present state	Condition	Next state	Outputs (UP DOWN GREEN BEACON)
S0 CLOSED (00)	R=0 / R=1	S0 / S1	0 0 0 0 (RED on)
S1 OPENING (01)	L=0 / L=1	S1 / S2	1 0 0 1
S2 OPEN (10)	P=1 / P=0	S2 / S3	0 0 1 0
S3 CLOSING (11)	P=1 | P=0,L=1 | P=0,L=0	S1 | S0 | S3	0 1 0 1

(c) Combinational logic (from K-map / SOP minimisation; ′ = NOT):
D1 = Q1·Q0′ + Q1′·Q0·L + Q1·Q0·P′·L′ **D0** = Q1′·Q0′·R + Q0·L′ + Q1·Q0′·P′ + Q1·Q0·P
UP = Q1′·Q0 **DOWN** = Q1·Q0 **GREEN** = Q1·Q0′ **RED** = Q1′·Q0′ **BEACON** = Q0

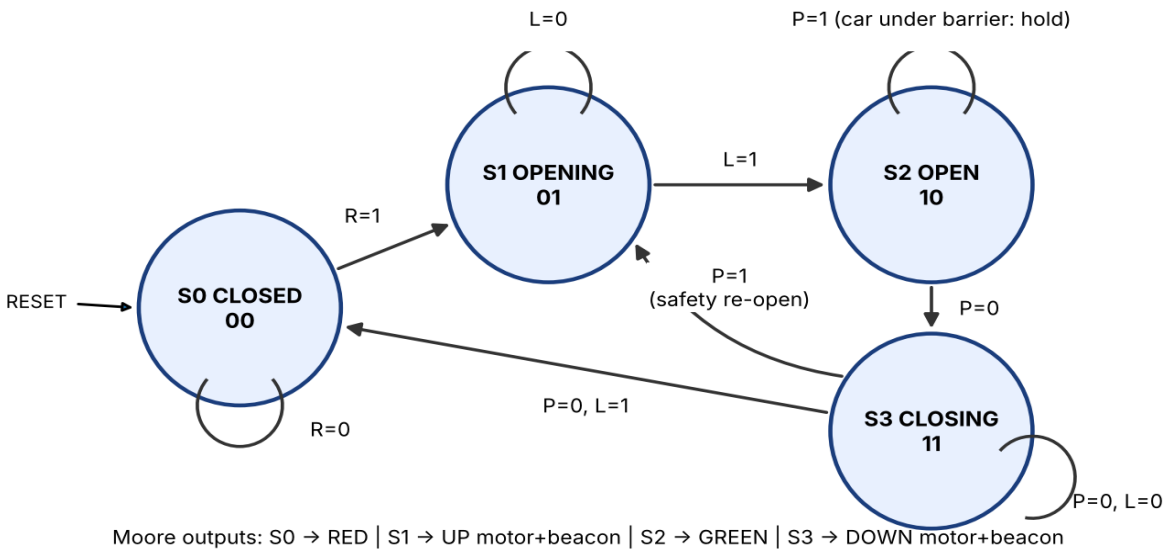
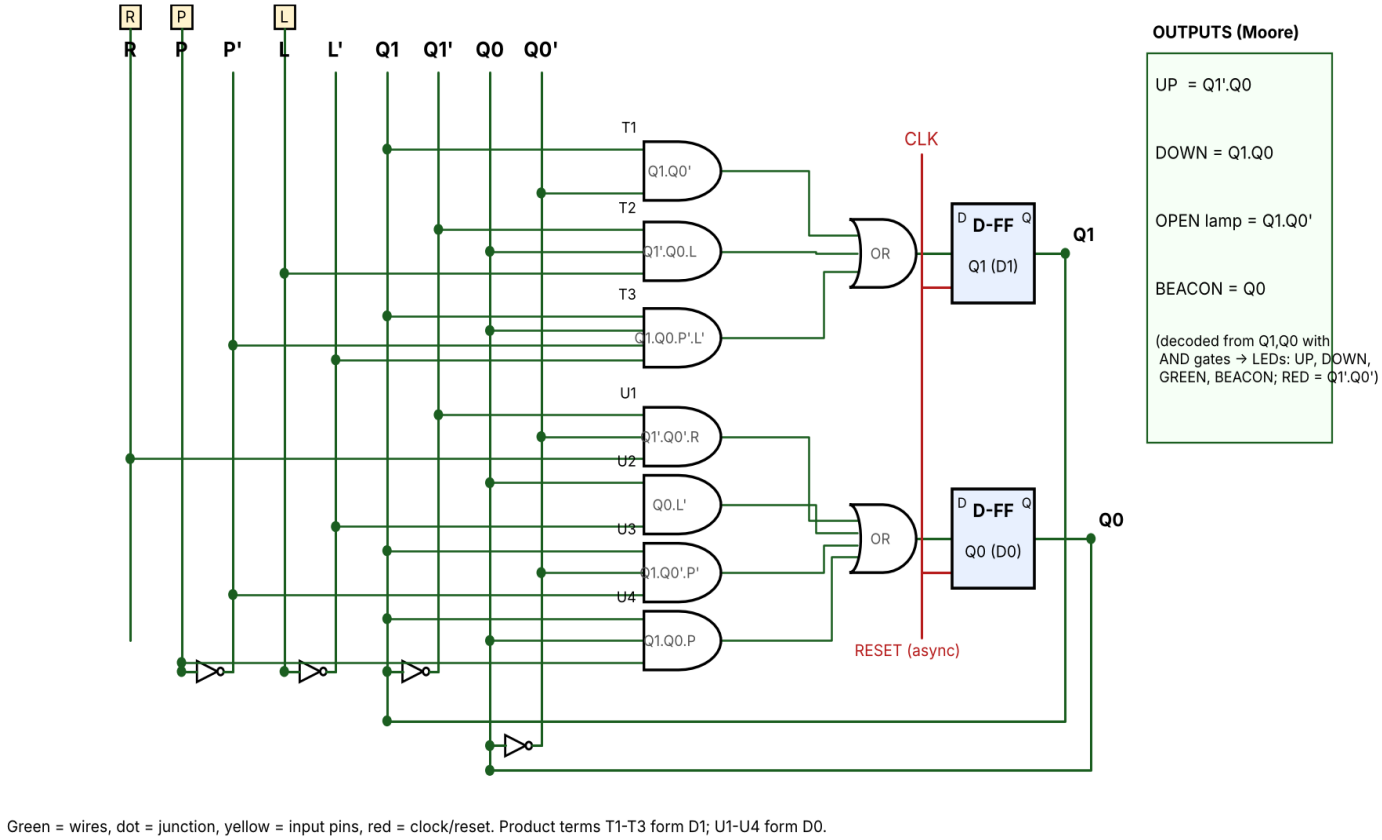


Fig. 1 - State diagram of the barrier controller

3. Logisim Implementation

Components: 3 input pins (R, P, L), 1 clock, 1 reset button, 2 D flip-flops (Q1, Q0), 4 NOT gates (P′, L′, Q1′, Q0′), 7 AND gates (T1-T3 for D1; U1-U4 for D0), 2 OR gates (3-input and 4-input), 4 decoding gates / LEDs for UP, DOWN, GREEN, BEACON (+ RED). **Steps:** (i) place inputs and flip-flops, set gate input counts; (ii) wire each product term to the Q/Q′ and input rails; (iii) OR the terms into D1 and D0; (iv) feed Q1, Q0 back to the rails; (v) connect CLK to both flip-flops and reset to their asynchronous clear; (vi) decode outputs and attach LEDs; (vii) run Simulate → Tick Enabled, or use Poke on R, P, L.



Green = wires, dot = junction, yellow = input pins, red = clock/reset. Product terms T1-T3 form D1; U1-U4 form D0.

Fig. 2 - Gate-level schematic of the controller (Logisim/CircuitVerse layout)

4. Result

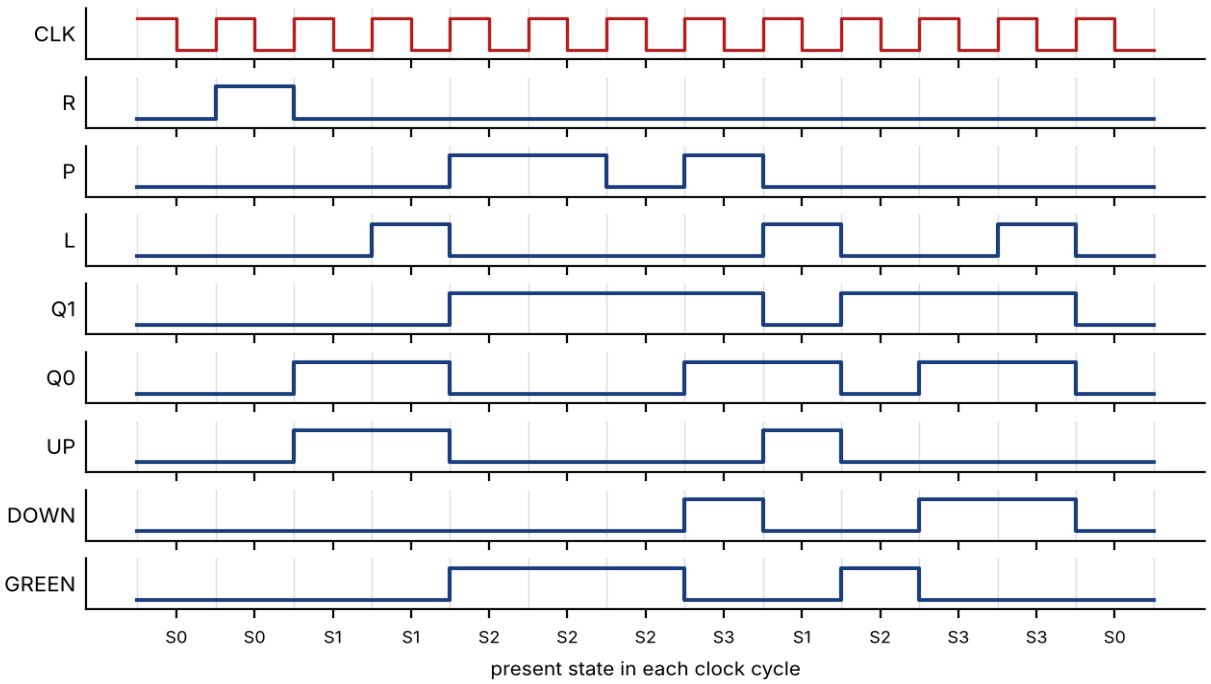


Fig. 3 - Timing waveform: request → open → vehicle passes → close → safety re-open → close

Exhaustive test - all 4 states × 8 input combinations (RPL) = 32 cases; next state from the implemented gate equations (matches specification in every case):

RPL	S0 (00)	S1 (01)	S2 (10)	S3 (11)
000	S0 (00)	S1 (01)	S3 (11)	S3 (11)
001	S0 (00)	S2 (10)	S3 (11)	S0 (00)
010	S0 (00)	S1 (01)	S2 (10)	S1 (01)
011	S0 (00)	S2 (10)	S2 (10)	S1 (01)
100	S1 (01)	S1 (01)	S3 (11)	S3 (11)
101	S1 (01)	S2 (10)	S3 (11)	S0 (00)
110	S1 (01)	S1 (01)	S2 (10)	S1 (01)
111	S1 (01)	S2 (10)	S2 (10)	S1 (01)

Result: 32 / 32 cases passed (0 mismatches between equations and state table); UP and DOWN were never 1 together in any state.

Edge cases documented

Edge case	Behaviour of the design
R=1 while OPENING / OPEN / CLOSING	Ignored (R only used in S0); no restart or glitch
P=1 and R=1 together while closed	Barrier opens (P is not used in S0)
P=1 in S3 and L=1 together (sensor conflict)	Safety wins: goes to S1 (re-open), arm never closes on a vehicle
Vehicle sits under open barrier	S2 holds as long as P=1; closing starts only when P=0
L stuck at 1 while OPENING	Goes to S2 immediately; stuck L at 0 keeps motor in S1 → external motor-timeout recommended
Power-up / unknown state	Asynchronous RESET forces S0; all 4 codes are used, so no unused / lock-up states
Input changes between clock edges	Ignored - synchronous design samples inputs only at the rising edge

5. Conclusion

A parking-barrier controller was specified, reduced to a 4-state Moore FSM with two D flip-flops, and implemented with simple AND-OR-NOT logic in Logisim/CircuitVerse. Combinational logic (next-state and output decoding) together with sequential logic (state memory) gave correct, glitch-free control: the barrier opens only on a valid request, holds while a vehicle is present, and re-opens if a vehicle enters the closing path. Exhaustive testing of all 32 state/input combinations and the documented edge cases verified the design, meeting CO2 (design of combinational and sequential circuits for a real application). The GenAI-derived table and equations were cross-checked by simulation before use.

4-bit ALU + Program Counter: Design, Simulation and Code-to-Hardware Mapping

CO4 (primary) | CO1 (reinforced)

1. Prompt

"Design and simulate, in Logisim, a 4-bit ALU made of a 4-bit adder/subtractor whose operation (+ or −) is chosen by a mode-select MUX, and a 4-bit up-counter that acts as a program counter, built from flip-flops with enable and reset inputs. Show the simulation results for both blocks. Then take a C statement such as `c = a + b;` and a loop `for(i=0;i<8;i++)`, and explain which hardware block performs the arithmetic (ALU), which holds the operand values (registers / flip-flops), and which sequences the instructions (program counter incrementing each iteration). Present the mapping as a labelled diagram with a short write-up, and finish with results and a conclusion."

2. Prompt Response (design and reasoning)

ALU. Four full adders are chained (ripple carry). A 2:1 MUX in front of each B input passes **B** when mode M=0 and **~B** when M=1; M is also fed to the first carry-in. So **M=0: S = A + B** and **M=1: S = A + ~B + 1 = A − B** (two's complement). Outputs: S3..S0, Cout (carry / no-borrow) and overflow V = C3 ⊕ Cout. Full adder: Sum = A⊕B⊕Cin, Cout = AB + Cin(A⊕B).
Program counter. Four D flip-flops share CLK, with an asynchronous RST (to 0000) and an enable EN. Each flip-flop input is $D_i = Q_i \oplus (EN \cdot Q_0 \cdot \ldots \cdot Q_{i-1})$, so the count increases by 1 on each clock edge when EN=1 and holds when EN=0. The loop condition i<8 is simply **Q3 = 0**, so EN = NOT Q3 ends the loop when the count reaches 1000.
Logisim build steps. (a) ALU: Wiring → two 4-bit inputs A, B, 1-bit input M; use four XOR gates (a 2:1 MUX of B/~B) or Plexers → Multiplexer; place Arithmetic → Adder (width 4) with Carry-In = M; add outputs S, Cout. (b) Counter: four Memory → D Flip-Flop, XOR + AND gates as in Fig. 2, inputs Clock, EN, RST; use Simulate → Ticks Enabled. A built-in Register + Adder (+1) gives the same behaviour.

3. Logisim Implementation

Fig. 1 4-bit ALU (Logisim layout): MUX(B / ~B) + 4 ripple full adders

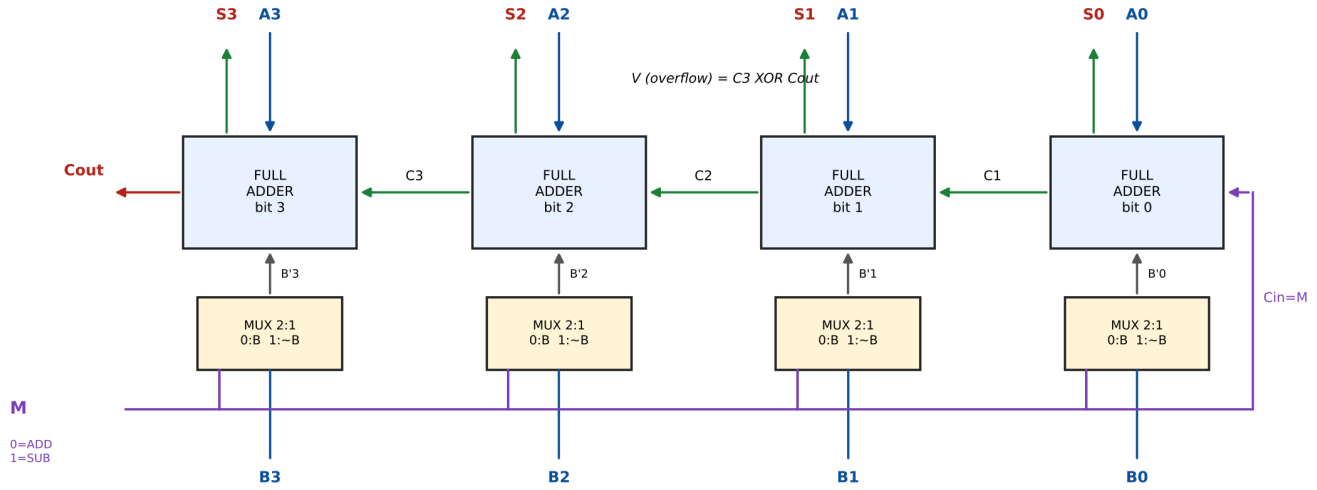
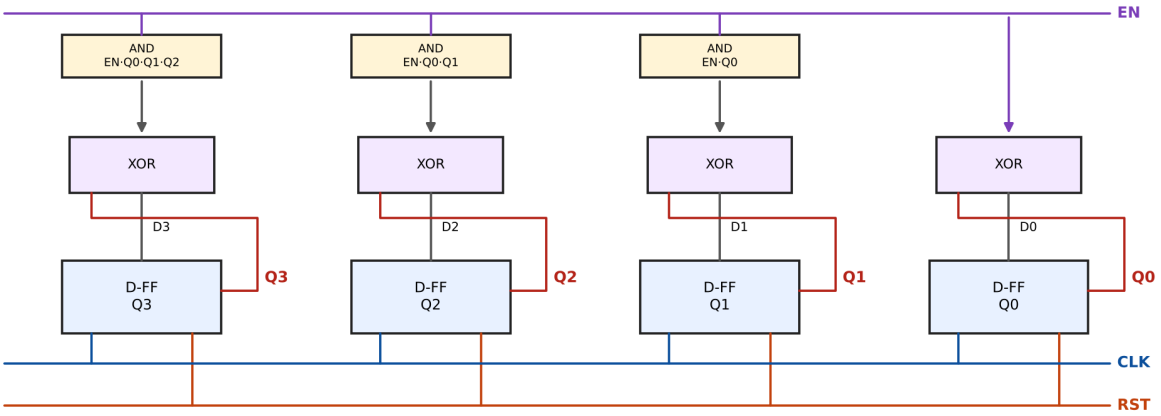


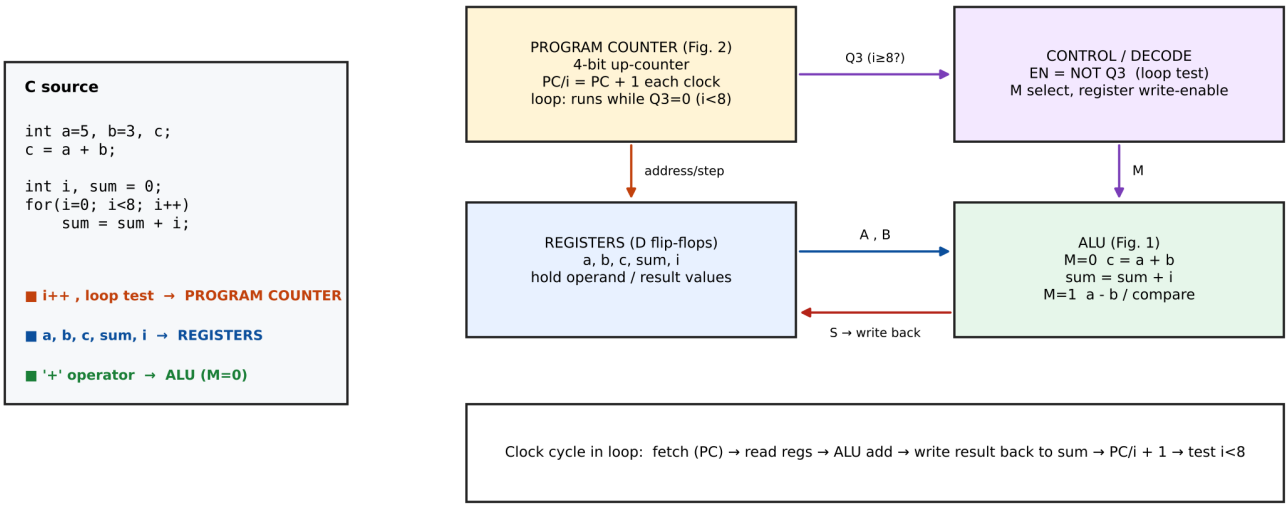
Fig. 2 4-bit program counter: D flip-flops with enable (EN) and asynchronous reset (RST)



Outputs Q3 Q2 Q1 Q0 = PC value | Next state: D = Q XOR (EN · lower Q bits) ⇒ Q+1 when EN=1, hold when EN=0, 0000 when RST=1

Code-to-hardware mapping — `c = a + b;` → **a, b** are read from **registers (D flip-flops)**, the **ALU** adds them with M=0, and the sum is written back into **c**. `for(i=0;i<8;i++)` → **i** is the up-counter/**program counter**, which steps once per iteration; `sum = sum + i` uses the ALU again; the test i<8 is the Q3 line.

Fig. 3 Mapping of `c = a + b;` and `for(i=0;i<8;i++) sum = sum + i;` onto hardware blocks



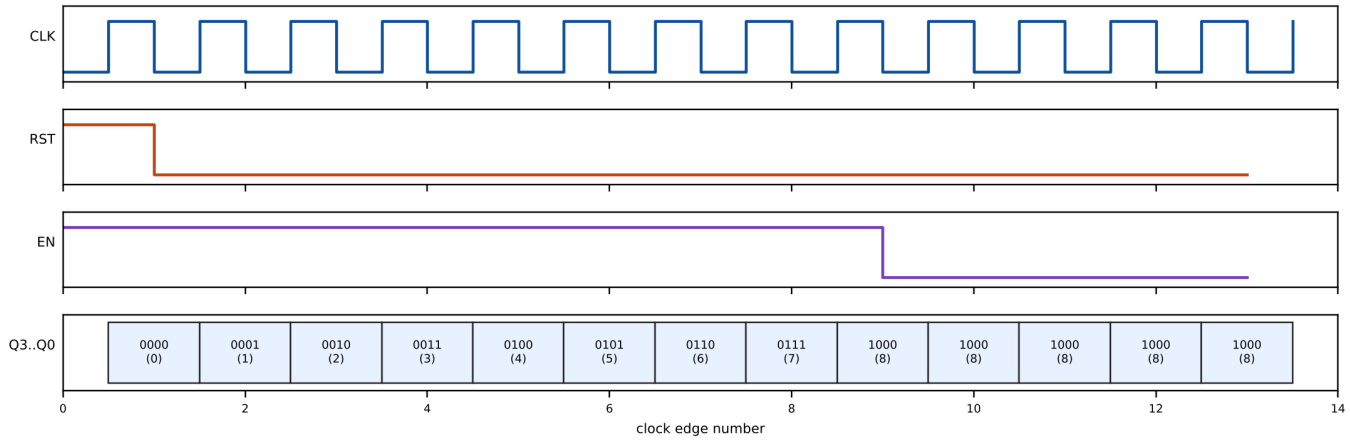
4. Result

A	B	M	Operation	S (bin)	S (dec)	Cout	V
0101	0011	0	A + B	1000	8	0	1
1001	0110	0	A + B	1111	15	0	0
0111	1001	0	A + B	0000	0	1	0
1100	0111	0	A + B	0011	3	1	0
0110	0010	1	A - B	0100	4	1	0
0011	0101	1	A - B	1110	14	0	0
0000	0001	1	A - B	1111	15	0	0
1000	0001	1	A - B	0111	7	1	1

i	sum before	ALU: sum+i	sum after	carry
0	0000 (0)	+0000	0000 (0)	0
1	0000 (0)	+0001	0001 (1)	0
2	0001 (1)	+0010	0011 (3)	0
3	0011 (3)	+0011	0110 (6)	0
4	0110 (6)	+0100	1010 (10)	0
5	1010 (10)	+0101	1111 (15)	0
6	1111 (15)	+0110	0101 (5)	1
7	0101 (5)	+0111	1100 (12)	0

Left: ALU test vectors (5+3=8; 9+6=15; 7+9=16 → 0 with Cout=1; 6–2=4; 3–5=14 i.e. –2 with Cout=0 = borrow; 0–1=15 i.e. –1). Right: `sum += i` for i=0..7 on the 4-bit ALU; the true total 28 wraps to 12 (1100) because the datapath is 4 bits wide.

Fig. 4 Counter timing: reset, count 0→8, then EN = NOT Q3 stops the loop (i=8 → exit)



Counter result: RST forces 0000; then Q increments 0001, 0010 ... 0111, 1000 on successive clock edges; at 1000, Q3=1 makes EN=0 and the value holds, which is the exit of `i<8`. Exactly 8 increments occur, matching 8 loop iterations.

5. Conclusion

A 4-bit ALU (MUX-controlled B/~B, ripple adders, Cin=M) correctly performs addition and two's-complement subtraction, and a flip-flop counter with enable and reset behaves as a program counter. For `c = a + b;` the registers supply operands, the **ALU** computes, and the result returns to a register; for the `for` loop the **counter/PC** sequences the 8 iterations while the ALU repeats the addition each cycle. This shows how high-level statements reduce to datapath (registers + ALU) and control (counter) hardware (CO4), built from basic gates and flip-flops (CO1). Limits: 4-bit width overflows (28 → 12), so wider registers are needed for larger values.

AI-Assisted Hardware Realization of Code: Comparator and Multiplexer Design

Course Outcomes: CO4 (primary), CO2 (reinforced). **Tools:** GenAI assistant, Logisim, Python (truth-table check).

GitHub repository: [paste link here]

1. Prompt

Given this short piece of code operating on 2-bit unsigned inputs a and b:

```
if (a > b) c = a; else c = b;
```

Propose a digital hardware realization of this logic using building blocks such as comparators, multiplexers, adders, counters and registers. Explain the chosen blocks, give both the block-level and gate-level structure, describe how the circuit can be simulated, and state how its truth table should be checked against the code's logic for all input combinations. Finally, point out any part of your own proposal that is incomplete or needs correction.

2. Prompt Response

Chosen blocks: The condition a > b is a magnitude comparison, so a 2-bit comparator produces a select signal S. The assignment c = a or c = b is a 2-to-1 selection, so two 2-to-1 multiplexers (one per bit) driven by S produce c. An adder, counter or register is not needed because there is no arithmetic and no state.

Gate-level comparator: With a = a1a0 and b = b1b0, the output is S = (a1 AND NOT b1) OR (NOT(a1 XOR b1) AND a0 AND NOT b0). The first term catches a1 > b1; the second catches equal high bits with a0 > b0.

Multiplexers: c1 = S ? a1 : b1 and c0 = S ? a0 : b0. Equality needs no special case because when a = b, both branches give the same value.

Simulation: Build the circuit in Logisim, drive all 16 input combinations, and compare the outputs with the code's result. Python was also used to check the same gate equations exhaustively.

3. Logisim Implementation

The design was built in Logisim with: four input pins (a1, a0, b1, b0), gates for the comparator equation (NOT, AND, OR, XOR), one S wire, two built-in 2-to-1 multiplexers with S as select, and two output pins (c1, c0). The block-level structure is shown in Figure 1.

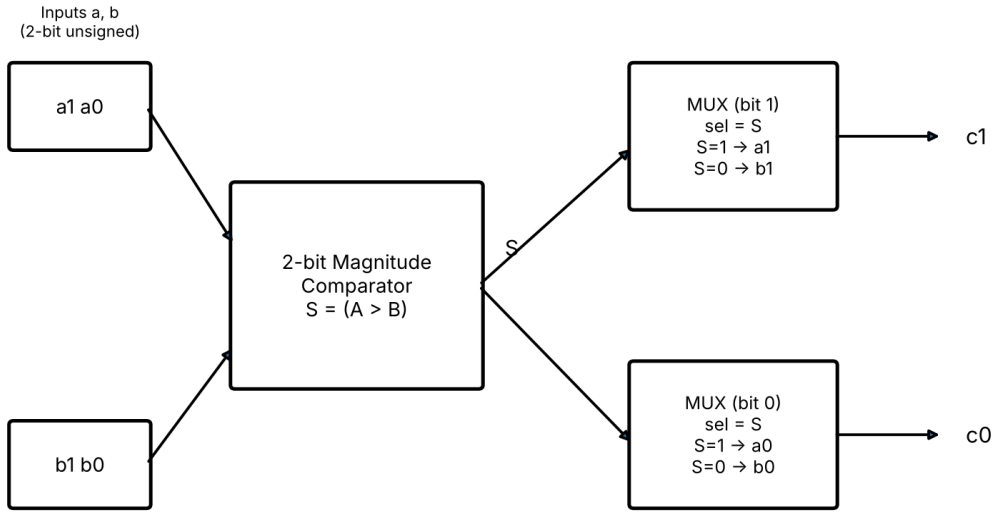


Figure 1. Block-level realization of if (a>b) c=a; else c=b;

Figure 1. Block-level realization of if (a>b) c=a; else c=b; (2-bit).

4. Result

Each of the 16 input combinations was evaluated through the gate equations and the multiplexers. The table below lists the outputs and the expected value from the code. All 16 rows match.

a	b	S(a>b)	c1	c0	c (circuit)	c (code)	Match
00	00	0	0	0	0	0	Yes
00	01	0	0	1	1	1	Yes
00	10	0	1	0	2	2	Yes
00	11	0	1	1	3	3	Yes
01	00	1	0	1	1	1	Yes
01	01	0	0	1	1	1	Yes
01	10	0	1	0	2	2	Yes
01	11	0	1	1	3	3	Yes
10	00	1	1	0	2	2	Yes
10	01	1	1	0	2	2	Yes
10	10	0	1	0	2	2	Yes
10	11	0	1	1	3	3	Yes
11	00	1	1	1	3	3	Yes
11	01	1	1	1	3	3	Yes
11	10	1	1	1	3	3	Yes
11	11	0	1	1	3	3	Yes

Reflection on the GenAI proposal: The first suggestion from the assistant was a full 2-bit subtractor followed by the sign bit as the select. That works but uses far more gates than needed, because only the comparison result matters. The correction was to use a dedicated magnitude comparator equation. The suggestion also did not mention that the two multiplexers must share one select line, so it was corrected in the Logisim wiring. It also gave no test set, so the exhaustive 16-row check was added by hand.

5. Conclusion

The statement if (a>b) c=a; else c=b; maps directly to one 2-bit magnitude comparator and two 2-to-1 multiplexers, with no adder, counter or register needed. The gate-level equation for the comparator and the multiplexer behaviour were verified over all 16 input combinations. The GenAI tool gave a correct direction but an over-complicated first design, which was fixed by checking against the truth table. This shows that AI-assisted design is useful for proposing structures, while verification remains the engineer's job.